

UnoArena

Plataforma de Uno en tiempo real y torneos masivos

Entrega final — arquitectura y decisiones

El dominio

- **Casual:** salas de 2–4 jugadores, una partida de Uno con todas sus reglas — comodines, +2 , +4 , el llamado de **UNO!** y el desafío.
- **Torneos:** N jugadores, rondas, mejor de tres, un campeón.
- **Espectadores:** cualquiera mira una sala en vivo — y **nunca** ve una mano.
- **Ranking y analytics:** Elo sobre partidas casuales, proyecciones de lectura.

El cliente es la CLI de la cátedra: es la que maneja el sistema.

Arquitectura final — diez desplegables

gateway	la única puerta: valida el token, enruta, sirve el SSE
identity	cuentas, sesión única activa, JWT
room-gameplay	el núcleo event-sourced: salas, jugadas, el log inmutable
outbox-relay	drena el outbox a Kafka como CloudEvents (corre dos veces)
timer-worker	el reloj detrás de los deadlines durables
tournament	el orquestador: log propio, outbox propio, saga, reconciliador
ranking	Elo + rating de posición

Una sola puerta

- Todo lo externo entra por `gateway` en **30080**. `identity` y `room-gameplay` son `ClusterIP` : no se alcanzan desde afuera.
- `room-gameplay` **no tiene la clave de firma**. Confía en `X-Player-Id` y `X-Session-Id` , que el gateway **sobrescribe en cada request**.

Ese "sobrescribe" es el control sobre el que se apoya todo el límite de confianza: un cliente no puede aportar sus propias cabeceras.

El log es la autoridad

Cada jugada aceptada se **appendea a** `room_events` **antes** de que nadie vea el resultado,
en la **misma transacción** que las filas del outbox.

- Si un agregado reconstruido discrepa del estado servido, **gana el log**.
- `publicPayload(event)` saca la semilla del RNG **dentro de esa transacción** — para cuando algo llega a Kafka, el mazo ya no está.

```
decide(state, command) → events    evolve(state, event) → state
```

La espina asíncrona

- **Kafka** con envoltorio **CloudEvents** (`ce-type` es una URI reverse-DNS, no el nombre).
- **Al menos una vez**, en orden por sala — el relay drena en orden de `id` .
- **Seis grupos de consumidores, tres pares de contrato** verificados en CI.
- El stream de **Redis** es un segundo camino, transitorio, para el feed en vivo. Nunca se construye un consumidor sobre él: **el durable es Kafka**.

Torneos: coreografía **y** una transacción

- El avance de ronda **es** una saga coreografiada: la sala decide el mejor de tres y publica `MatchCompleted` con `advancingPlayers` .
- Pero la **provisión de salas es síncrona**, por `POST /internal/rooms` .

Si las salas se crearan por evento, `RoundStarted` sería una promesa: nombraría salas

que quizá todavía no existen. Creándolas antes, **no puede** nombrar una sala que nadie creó.

Entrega: construir una vez, promover por digest

```
test → build → deliver → deploy-staging → integration-staging → (prod)
```

- **Detección de cambios por path:** tocar un servicio corre **solo** ese servicio.
- `deliver` captura el `@sha256:...`; un commit del bot lo escribe en el overlay. Producción recibe **el mismo digest** — nunca un rebuild.
- **GitOps con Argo CD:** el pipeline nunca hace `kubectl apply`. Escribe git; Argo reconcilia.

Observabilidad — del mismo install

- **Tres tableros como JSON commiteado:** negocio, golden signals, espina asíncrona.
- **Nueve reglas de alerta;** cada una es una falla que este proyecto tuvo de verdad.
- **Loki + Alloy:** un `correlationId` devuelve **cinco servicios** en una query.

Las tres métricas de negocio — nombradas en el tablero mismo:

```
roomgameplay_games_completed_total .
```

```
tournament_tournaments_completed_total .
```

```
identity_registrations_total
```

Los números

Desplegables reales	10 (ninguno con <code>digest: ""</code>)
Aplicaciones de Argo	24
De cluster vacío a 24/24	12 m 25 s (<i>kind, caché tibia</i>)
Targets de scrape	11
Servicios en una query de <code>correlationId</code>	5
Pipeline más ancho	43 jobs — 42 success + 1 manual
Eventos que publica un demo entero	179

Lo que **no** hicimos

- **Sin backend de tracing.** Los logs con `correlationId` responden la misma pregunta acá; los spans de OpenTelemetry quedan como opción declinada.
- **Siete de nueve reglas de alerta nunca se vieron disparar** — están listadas como no probadas, no como cobertura.
- **Sin receivers de alertas.** El plano de alerta existe; nadie paginea un canal que nadie lee.
- **Sin almacenamiento persistente** para observabilidad.

Todo esto está escrito en el repo, no dicho acá por primera vez.

Qué haríamos después

- **Receivers y SLOs**: las alertas existen; falta a quién avisarle y contra qué objetivo.
- **Tracing distribuido**, cuando haya más de un salto asíncrono que seguir.
- **Escalar `ranking`**: sus dos escrituras de rating ya son seguras ante concurrencia, así que las réplicas volvieron a ser una decisión de capacidad.
- **Multi-región** — hoy explícitamente fuera de alcance.

Gracias

Repo: `gitlab.com/itba-73-40-microservicios/alumnos/2026-s1/grupo-4/amicro-tp`

Runbook del demo: `docs/demo-runbook.md`

Runbook de observabilidad: `docs/observability-runbook.md`

Desviaciones de diseño: `CHANGELOG-design.md`